

OpenID Connect (OIDC)

Key Concepts

- **OIDC** — identity layer built on top of OAuth 2.0; adds authentication to OAuth's authorization
- **ID Token** — a JWT containing who the user is (identity claims); issued alongside the Access Token
- **openid scope** — the trigger; including this in the OAuth request tells the Auth Server to issue an ID Token
- **OAuth 2.0 answers:** "What can this app access?"
- **OIDC answers:** "Who is this user?"

OAuth 2.0 vs OIDC

	OAuth 2.0	OpenID Connect
Type	Authorization framework	Authentication layer on OAuth
Answers	What can the app access?	Who is the user?
Token issued	Access Token	Access Token + ID Token
Use case	Allow app to read your Google Drive	Sign in with Google

How It Works

Same as OAuth 2.0 Authorization Code Flow, with two differences:

1. **scope** includes **openid**:

```
scope=openid email profile
```

2. Token response includes an ID Token:

```
{
  "access_token": "ya29.a0...",
  "id_token": "eyJhbGc...",
  "refresh_token": "1//0g...",
  "expires_in": 3600
}
```

ID Token Structure (JWT)

```
{
  "sub": "1234567890",
  "name": "Anil Bhasi",
  ...
}
```

```
{  "email": "anil@gmail.com",  "picture": "https://...",  "iss": "https://accounts.google.com",  "aud": "spotify_client_id",  "exp": 1716000000,  "iat": 1715996400}
```

Claim	Meaning
<code>sub</code>	Unique user ID at the identity provider
<code>iss</code>	Issuer — who created this token
<code>aud</code>	Audience — which client this token is for
<code>exp</code>	Expiry
<code>name</code> , <code>email</code>	Standard identity claims

Key Point

- Client reads the **ID Token** to know who the user is — no extra API call needed
- Client uses the **Access Token** to call APIs (Google Drive, Gmail, etc.)
- The `sub` claim is the stable unique identifier — use this as the user's ID in your DB, not email (emails can change)

Industry Examples

- **"Sign in with Google"** — OIDC; returns ID Token with user identity
- **Auth0 / Okta** — OIDC providers; issue ID Tokens for SSO across enterprise apps
- **Microsoft Azure AD** — OIDC for enterprise identity federation
- **Apple Sign In** — OIDC-based

Interview Questions

Q: What is OpenID Connect and how does it differ from OAuth 2.0?

OAuth 2.0 is an authorization framework — it grants access to resources but doesn't identify the user. OIDC sits on top of OAuth and adds authentication by issuing a signed ID Token (a JWT) containing the user's identity. Including `scope=openid` in the request triggers the ID Token.

Q: What is the ID Token and what does it contain?

A JWT issued alongside the Access Token. Contains identity claims: `sub` (user ID), `email`, `name`, `iss` (issuer), `aud` (audience), `exp` (expiry). Client reads it to know who logged in — no extra API call needed.

Q: Why use `sub` instead of `email` as the user identifier?

`sub` is a stable, immutable identifier at the identity provider. Email addresses can change. Always store `sub` + `iss` together as the unique key for a user.

Q: Can you use OAuth without OIDC?

Yes — OAuth alone is fine when you only need to access resources (call APIs) on behalf of a user. OIDC is needed when you also need to know who the user is (login/signup flows).